Клеточные автоматы (Cellular Automata)

Клеточные автоматы (Cellular Automata)

Идея

Помните наивный подход с хаб-страницы? Мы заполняли матрицу случайно и получали бесполезный шум. А что, если этот шум - не конечный результат, а заготовка? Что, если мы возьмём его и "сгладим"?

Ну, в этом и суть клеточных автоматов для генерации пещер. Мы берём тот самый случайный шум и несколько раз пропускаем его через простое правило: если вокруг ячейки много стен - она тоже становится стеной. Если мало - становится полом. После нескольких таких проходов хаос самоорганизуется в гладкие, естественно выглядящие пещеры.

Если вы слышали про "Игру Жизни" - принцип тот же. Набор ячеек, каждая из которых меняет своё состояние в зависимости от соседей. Только правила другие.

Правило сглаживания

У каждой ячейки в матрице есть 8 соседей (включая диагональные). Мы считаем, сколько из них - стены. И дальше применяем правило:

- Если у ячейки **5 и более** соседей-стен - она становится стеной.
- Если **меньше 5** - она становится полом.

Вот и весь секрет. Ок, почему это работает? Потому что стены, стоящие рядом друг с другом, "выживают" и образуют массивы. Одинокие стены, окружённые полом, исчезают. Одинокие пустоты, окружённые стенами, заполняются. В результате - чёткие границы между проходимыми областями и стенами.

Порог "5 из 8" - не единственный вариант. Можно поставить 4, можно 6. Можно 1, 20. Попробуйте разные значения и посмотрите, как меняется результат. Придумывайте свои правила.

Реализация

Алгоритм простой, но разберём по частям - собирать генератор целиком вы будете сами.

CountWallNeighbors - подсчёт соседей

Для каждой ячейки проверяем все 8 соседей (включая диагональные). Ячейки за пределами матрицы считаются стенами - это естественным образом создаёт стены по краям уровня.

Python C#

```python
import random

def count_wall_neighbors(grid, cx, cy, width, height):
    count = 0
    for dy in range(-1, 2):
        for dx in range(-1, 2):
            if dx == 0 and dy == 0:
                continue
            nx, ny = cx + dx, cy + dy
            # out of bounds counts as wall
            if nx < 0 or nx >= width or ny < 0 or ny >= height:
                count += 1
            elif grid[ny][nx] == 1:
                count += 1
    return count
```

Сглаживание - один проход

На каждой итерации создаём **новую** матрицу и заполняем по правилу: 5+ стен вокруг → стена, иначе → пол. Важно писать в новую матрицу, а не в текущую - иначе ячейки, обработанные раньше, исказят подсчёт соседей для остальных.

Python C#

```python
new_grid = [[0] * width for _ in range(height)]
for y in range(height):
    for x in range(width):
        walls = count_wall_neighbors(grid, x, y, width, height)
        new_grid[y][x] = 1 if walls >= 5 else 0
grid = new_grid
```

Осталось собрать: создайте матрицу `width × height`, заполните случайно (каждая ячейка с вероятностью `wall_chance` - стена, иначе - пол), затем прогоните сглаживание `iterations` раз.

Результат

Самое интересное - как меняется карта от итерации к итерации:

- **0 итераций**: чистый шум. То, что мы видели в наивном подходе.
- **1 итерация**: шум начинает группироваться. Появляются "сгустки" стен и пола, но границы рваные.
- **3 итерации**: чёткие контуры пещер. Большинство "мусора" исчезло.
- **5 итераций**: гладкие, стабильные пещеры. Дальнейшие итерации практически ничего не меняют.

Т.е., грубо говоря, алгоритм **сходится** - после определённого числа проходов карта перестаёт меняться. Обычно хватает 4-6 итераций.

Влияние параметров

- **wall_chance** (вероятность стены при заполнении): чем выше, тем больше стен в итоге. При 0.45-0.50 получается примерно поровну пещер и стен. При 0.55+ пещеры становятся маленькими и изолированными. При 0.35- стен почти не остаётся.
- **iterations** (число проходов): больше проходов - более гладкий результат. Но после 5-6 разница минимальна.
- **Порог** (5 в нашем примере): при пороге 4 пещеры получаются более открытые, при 6 - более узкие.

Проблема связности

Хм, есть важный нюанс. В отличие от случайного блуждания, где один строитель гарантирует связность, клеточные автоматы **не дают такой гарантии**. На карте могут образоваться несколько отдельных пещер, не соединённых друг с другом.

Что с этим делать?

- **Проверить связность** через заливку и перегенерировать, если уровень непроходим.

- **Соединить регионы**: найти изолированные области заливкой, затем прорезать коридор между ближайшими точками двух крупнейших регионов.
- **Удалить мелкие регионы**: если в результате генерации есть одна большая пещера и несколько маленьких "островков" пола, можно просто залить маленькие стенами. Или наоборот - мелкие изолированные "столбы" стен внутри пещеры можно убрать.

Все три варианта начинаются с одного: найти регионы.

Поиск регионов (BFS)

Проходим по всей матрице, ищем непосещённую ячейку пола. Нашли - запускаем BFS по четырём направлениям (вверх, вниз, влево, вправо), собирая все связанные ячейки пола в один регион. Помечаем посещённые. Повторяем, пока не обойдём всю матрицу. Результат - список регионов, каждый - список координат `(x, y)`.

Удаление мелких регионов

Имея список регионов, проходим по каждому: если регион меньше `min_size` ячеек - заливаем стенами. Мелкие изолированные "столбы" стен внутри большой пещеры тоже можно убрать аналогично - только искать регионы стен, а не пола.

Для подсчёта соседей при сглаживании мы проверяем все 8 направлений (включая диагонали), а при поиске регионов - только 4. Стена по диагонали влияет на "характер" ячейки, но пройти по диагонали в тайловой игре нельзя - связность проверяем только по сторонам.

Комбинирование подходов

Клеточные автоматы хорошо работают в связке с другими алгоритмами. Например:

- **Случайное блуждание + сглаживание**: сначала прорезаем основную структуру случайным блужданием (это гарантирует связность), а потом "сглаживаем" результат одним-двумя проходами клеточного автомата. Получаются связные пещеры с гладкими стенами.
- **Автоматы для отдельных комнат**: если вы генерируете подземелье с помощью BSP, можно заполнять отдельные комнаты через клеточные автоматы, чтобы они выглядели как естественные пещеры, а не прямоугольники.

Плюсы и минусы

- **Компактность**: алгоритм очень короткий. Вся суть - в одной функции подсчёта соседей и одном цикле.
- **Визуальное качество**: результат выглядит естественно. Гладкие стены, плавные изгибы, никакой "пиксельной каши".
- **Настраиваемость**: три параметра (wall_chance, iterations, порог) дают достаточно контроля. Легко подобрать нужный "характер" пещер.

Но:

- **Нет гарантии связности**: могут появиться изолированные регионы. Нужна постобработка.

- **Нет структуры**: как и случайное блуждание, этот алгоритм не создаёт комнат и коридоров. Только пещеры.
- **Вариативность результата**: при одних и тех же параметрах могут получаться как отличные, так и неудачные карты. Seed помогает с воспроизводимостью, но не с качеством.

Что дальше?

- Случайное блуждание - если вы ещё не смотрели. Хорошо комбинируется с клеточными автоматами.
- BSP-генерация - если пещеры - не то, что вам нужно, и вы хотите комнаты с коридорами.
- Шум Перлина - для ландшафта и открытых миров.
- Wave Function Collapse - генерация из правил соседства.
- L-системы - формальные грамматики для растений и фракталов.

Powered by Wiki.js